

Funciones

Una función es un mini programa dentro de un programa.

Las funciones:

- Contienen varias sentencias bajo un solo nombre.
- Proporcionan un medio de dividir un proyecto grande en módulos pequeños.
- Existen de modo autónomo, cada una tiene su ámbito.

La división del código en funciones hace que las mismas se puedan reutilizar en su programa y en otros programas.

Si se agrupan funciones en bibliotecas otros programas pueden utilizar estas funciones

Una función proporciona una forma conveniente de encapsular algunos cálculos, que se pueden emplear después sin preocuparse de su implementación.

Una función es un subprograma que se llama dentro de un programa principal. Las funciones son independientes del programa que las llama, por eso una función puede ser utilizada varias veces dentro de un programa o ser utilizada dentro de otros programas.

Una **definición de función** tiene la siguiente forma:

def nombre de la función(param1, param2, ...):

Los **parámetros o parámetros formales** representan los nombres de los elementos que se transfieren a la función desde la parte del programa que las llama. Una función puede no tener parámetros

```
def mostrarmensaje():  
    print("hola")  
    print("mundo")
```

Sólo muestra el mensaje "hola mundo" cuando se la llama.

La **estructura de función** es:

```
def nombre (lista de parámetros):  
    cuerpo de la función  
    return expresión
```

Lista de parámetros: se enumeran los parámetros en un cierto orden que debe ser el mismo todas las veces que se utiliza la función: parámetro1, parámetro2.

Cuerpo de la función: se indentan las líneas de código que corresponden a la función.

Valor devuelto por la función: mediante la palabra **return** se regresa el valor de la función al programa principal. Este valor debe ser único. Si la función no regresa ningún valor, puede no escribirse la palabra return.

La llamada a una función produce la ejecución de las sentencias del cuerpo de la función y un retorno a la unidad de programa llamadora después que la ejecución de la función se ha terminado.

Ejemplo 1:

Dados dos números, mostrar el mayor (los números son distintos)

```
def mayor(a, b):  
    if a > b:  
        print("elmayor es ", a)  
    else:  
        print("elmayor es ", a)  
  
//Programa principal  
x = int(input("ingrese un valor"))  
y = int(input("ingrese un valor"))  
mayor(x, y)
```

En las variables a y b de la función se guardan los valores de c e y que se ingresan en el programa principal.

Esta función imprime cuál es el mayor y no devuelve ningún valor al programa principal.

Ejemplo 2:

Dado un número natural mostrar su factorial.

```
def factorial(n):  
    h = 1  
    for i in range(1, n+1, 1):  
        h = h * i  
    return h  
  
//Programa principal  
x = int(input("ingrese un valor"))  
f = factorial(x)  
print("el factorial es", f)
```

La función devuelve el valor del factorial (h) que se asigna a la variable f, y luego se muestra ese valor en el programa principal.

Ejemplo 3:

Calcular la suma de n números enteros.

```
def suma(n):  
    a = 0  
    for i in range(0, n, 1)  
        x = int(input("ingrese un número"))  
        a = a + x  
    print ("la suma es", a)  
    return
```

```
x = int(eval(input("ingrese la cantidad de números que quiere sumar")))  
suma (x)
```

¿Cómo se utilizan las funciones en un programa?

Si se utilizan funciones dentro de un programa, éstas aparecen en dos instancias:

- la primera es la **definición y desarrollo** antes del programa principal.
- la segunda es la **llamada** de la función dentro del programa principal.

Llamada a la función dentro del programa principal.

Las funciones para poder ser ejecutadas, han de ser llamadas.

La función llamada que recibe el control del programa se ejecuta desde el principio hasta el final y cuando termina devuelve el control al programa principal, a la línea siguiente donde fue llamada.

Cuando se llama a la función dentro del programa principal los argumentos se llaman argumentos reales, ya que definen la información real que se transfiere. Los argumentos reales son variables que se utilizan en el programa o valores constantes.

Si la función no regresa ningún valor en la llamada sólo se nombra la función.

Si la función regresa un valor, en la llamada debe haber una asignación del valor que regresa a otra variable del mismo tipo que es la que reconocerá el programa principal.

Definición y desarrollo

Antes de comenzar el programa principal se **desarrolla la función** con su definición y su cuerpo. A la definición de la función deben anteceder dos líneas en blanco. La función puede tener cualquier número de sentencias, pero deben estar indentadas desde el margen izquierdo. Podemos darnos cuenta que una determinada línea del cuerpo de la función es la última cuando la siguiente línea no está indentada.

Los parámetros que se utilizan en la definición de la función y en la llamada deben estar escritos en el mismo orden.

Los nombres de los parámetros, en la llamada y en la definición, pueden no coincidir.

La función que se invoca no puede alterar directamente una variable del programa principal, sólo puede modificar su copia temporal. Es decir, si entra una variable del programa principal y se modifica en la función, al volver el control al programa principal, la variable no llega modificada, salvo que su valor sea el que retorne la función a través del return.

Ejemplo:

Escribir a 2 columnas las potencias de 2 y de 3 con exponentes 0 al 9.

```
def power(m, n):  
    p = 1  
    for i in range(1, n, 1):  
        p = p * m  
    return p  
  
for i in range(0, 10, 1):  
    if i == 0:  
        a = 1  
        b = 1  
    else:  
        a = power(2, i)  
        b = power(3, i)  
    print("\t", i, a, b)
```

Cada llamada pasa 2 argumentos a **power()**, y cada vez regresa un entero **p**.

En el programa principal, los argumentos que se pasan a power() son una constante (la potencia) y el otro a través de una variable (la base).

En la definición o desarrollo de `power()` los nombres **m** y **n** son argumentos locales. Por el orden en que están, la **constante** se corresponde con **m** y la variable **i** con **n**.

```
a = power (2, i)
      ↓   ↓
def power (m, n)
```

Las variables **i** y **p** que se declaran en la función son variables **locales**, es decir variables que no se ven en el principal y que si no regresan como valor de la función pierden su valor al salir de la misma.

En nuestro ejemplo **p** regresa su valor por estar declarada en el `return` y en el principal su valor se asigna a las variables `a` o `b`, y como la variable **i** de la función no es devuelta al programa principal, no modifica el valor de la variable **i** del mismo.

Python, como otros lenguajes, tiene funciones ya definidas como `print()`, `len()`, `lower()` etc. Observemos que todas están seguidas de un paréntesis en el que se espera coloquemos los parámetros. En el *print* escribimos lo que queremos que aparezca en pantalla, en el *len* la cadena que queremos medir, etc.